

Checklist

CI/CD as a Platform Service

1. Assess the Current State

- Inventory existing CI/CD workflows across repositories
- Measure total workflow count and pipeline line count
- Identify repositories already using shared platform actions
- Establish baseline platform adoption rate
- Record deployment frequency
- Record lead time for changes
- Record change failure rate
- Record mean time to restore
- Record p95 build duration
- Record security scan pass rate

2. Create the Platform Actions Repository

- Create an `actions/` directory for composite actions
- Create a `workflows/` directory for reusable workflows
- Create a `tests/` directory for validation and integration tests
- Create a `docs/` directory for usage guides, migration instructions, and changelogs
- Keep reusable tasks separate from composed pipeline templates

3. Build Reusable Actions

- Create a reusable container-build action
- Configure Docker Buildx
- Generate container metadata and tags
- Configure build caching
- Add container registry authentication
- Add vulnerability scanning with Trivy
- Fail builds on HIGH or CRITICAL vulnerabilities

- Ensure vulnerable images are not pushed to the registry
- Expose useful outputs such as image tag and image digest
- Pin third-party actions to known versions

4. Test Platform Actions

- Validate `action.yml` syntax
- Verify required fields such as `name`, `description`, and `runs`
- Validate supported runner/action types
- Confirm composite actions contain steps
- Confirm every input has a description
- Run validation automatically in CI
- Prevent invalid platform actions from being released

5. Build Reusable Pipeline Templates

- Create reusable workflows for common application types
- Create a backend microservice pipeline
- Add typed workflow inputs
- Add sensible default values
- Configure secrets at the workflow level
- Add automated testing
- Add linting
- Add container build and security scanning
- Add deployment stages
- Configure job dependencies with `needs`
- Provide controlled escape hatches where appropriate
- Expose outputs required by downstream workflows

6. Keep Team Pipelines Minimal

- Replace custom team pipelines with references to platform templates
- Keep team-owned workflow files as minimal wrappers
- Target fewer than roughly 30 lines per team pipeline
- Let teams specify **what** they need
- Keep implementation details—the **how**—inside platform templates
- Investigate large amounts of team-specific logic as a possible missing platform capability

7. Version the Platform

- Apply semantic versioning to platform actions and workflows
- Use major versions for breaking changes
- Use minor versions for new compatible features
- Use patch versions for fixes
- Create specific release tags such as `v1.2.3`
- Maintain floating major tags such as `v1`
- Allow teams to pin to stable versions
- Document breaking changes and migration requirements

8. Implement Progressive Delivery

- Install Argo Rollouts
- Choose an appropriate deployment strategy
- Configure blue-green deployment where instant switching is useful
- Configure canary deployment where gradual traffic shifting is preferred
- Connect deployment analysis to Prometheus metrics
- Define success-rate thresholds
- Shift traffic progressively during canary releases
- Automatically stop or roll back unhealthy releases
- Verify the previous stable release remains recoverable

9. Add CI/CD Observability

- Deploy or configure an OpenTelemetry Collector
- Receive GitHub workflow events
- Collect workflow and job telemetry
- Export pipeline metrics to Prometheus
- Export traces to the tracing backend
- Track pipeline execution from trigger through deployment
- Measure time spent in tests, builds, and deployments
- Associate failures with commits or infrastructure changes
- Make pipeline traces visible in Grafana
- Use telemetry to identify pipeline bottlenecks

10. Migrate the Demo/Application Pipeline

- Update the application CI workflow to use the reusable platform pipeline

- Provide service-specific inputs
- Configure required secrets
- Pin the platform workflow to a stable version
- Update GitOps manifests with the generated image
- Verify Flux CD detects and applies the change
- Confirm Argo Rollouts starts the progressive deployment

11. Validate the Final Implementation

- Platform actions repository contains reusable tasks
- Application pipeline uses composed platform tasks
- Automated tests and security scans pass
- Canary or blue-green deployment functions correctly
- Automated rollback is verified
- Pipeline traces are visible in Grafana
- Platform adoption metrics are being collected
- DORA metrics can be measured
- Pipeline line-count reduction can be demonstrated
- Documentation and migration guidance are available for development teams